

本小节内容

字符数组初始化及传递

scanf 读取字符串

1 字符数组初始化及传递

字符数组的定义方法与前面介绍的一维数组类似。例如，

```
char c[10];
```

字符数组的初始化可以采用以下方式。

(1) 对每个字符单独赋值进行初始化。例如，

```
c[0]='l';c[1]=' ';c[2]='a';c[3]='m';c[4]=' ';c[5]='h';c[6]='a';c[7]='p';c[8]='p';c[9]='y';
```

(2) 对整个数组进行初始化。例如，

```
char c[10]={'l','a','m','h','a','p','p','y'}
```

但工作中一般不用以上两种初始化方式，因为字符数组一般用来存取字符串。通常采用的初始化方式是 `char c[10]= "hello"`。因为 C 语言规定字符串的结束标志为 `'\0'`，而系统会对字符串常量自动加一个 `'\0'`，为了保证处理方法一致，一般会人为地在字符数组中添加 `'\0'`，所以字符数组存储的字符串长度必须比字符数组少 1 字节。例如，`char c[10]` 最长存储 9 个字符，剩余的 1 个字符用来存储 `'\0'`。

【例】字符数组初始化及传递

```
#include <stdio.h>
```

```
void print(char c[])
```

```
{
```

```
    int i=0;
```

```
    while(c[i])
```

```
    {
```

```
        printf("%c",c[i]);
```

```
        i++;
```

```
    }
```

```

printf("\n");

}

//字符数组存储字符串，必须存储结束符'\0'

int main()

{

    char c[5]={'h','e','l','l','o'};

    char d[5]="how";

    printf("%s\n",c); //会发现打印了乱码

    printf("%s\n",d);

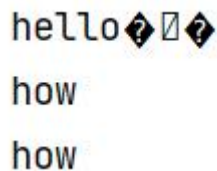
    print(d);

    return 0;

}

```

上例中代码的执行结果如下图所示。为什么对数组赋值"hello"却打印出乱码，这是因为printf通过%s打印字符串时，原理是依次输出每个字符，当读到结束符'\0'时，结束打印；



```

hello???
how
how

```

我们通过print函数模拟实现printf的%s打印效果，当c[i]为'\0'时，其值是0，循环结束，也可以写为c[i]!='\0'。

2 scanf 读取字符串

【例】scanf 读取字符串

```

#include <stdio.h>

//scanf 读取字符串时使用%s

```

```
int main()

{

    char c[10];

    char d[10];

    scanf("%s",c);

    printf("%s\n",c);

    scanf("%s%s",c,d);

    printf("c=%s,d=%s\n",c,d);

    return 0;

}
```

scanf 通过%s 读取字符串，对 c 和 d 分别输入"are"和"you"（中间加一个空格），**scanf** 在使用%s 读取字符串时，会忽略空格和回车（这一点与%d 和%f 类似）。

输入顺序及执行结果如下图，

```
hello
hello
are you
c=are,d=you
```